

SOLID 原则	检查问题	AI 设计是否违反	违反说明	修正方案
S - 单一职责	有没有类承担了过多职责？	是	原来的 Student 类承担了过多的业务线职责：它既是需求的发布方（买方逻辑），又是需求的承接方（卖方逻辑），同时还集合了信用分和评价。这导致任何一方业务变动都会影响该类。	将 Student 的基础属性收拢到符合最新底层数据库设计的 User 类中；将发单与接单的业务行为解耦，并拆分出独立的 Notification 类来单独负责通知职责。
O - 开闭原则	新增需求类型是否需要修改现有代码？	是	原来的 Requirement 依赖固定的类型枚举。当平台想新增新分类时，由于某些分类具有特异性（例如“组队匹配”不需要评价功能），必须修改原有的判断代码。	引入动态的 Category 实体类。将分类的行为（如 has_review 是否支持评价）数据化、配置化。后续增加新类型时，只需在数据库加记录，核心代码无需改动。
L - 里氏替换	子类是否可以替换父类使用？	否	\	\
I - 接口隔离	有没有接口太“胖”，包含了不需要的方法？	否	\	\
D - 依赖倒转	高层模块是否直接依赖了底层模块的具体实现？	是	高层业务模型(如 Requirement、Order)直接强耦合关联了底层的具体实现类 Student（或显式关联了具体 ID），导致高层逻辑无法脱离底层具体类独立存在和复用。	遵循依赖倒转原则，让高层模块不再直接依赖具体的 Student 类。使 Task（原 Requirement）与 Order 改为关联抽象的 IRequester 和 IProvider 接口。